Programación de servicios y procesos

UNIDAD 1

Multiprocesos

Profesor: Miguel Soriano – msoriano@ucam.edu

Ciclo Formativo de Grado Superior

FP SAN ANTONIO
INSTITUTO SUPERIOR DE FORMACIÓN PROFESIONAL

CONTENIDOS

1. Introducción a los procesos
2. Creación y gestión de procesos con Java
3. Comunicación entre procesos
4. Sincronización y obtención de resultados de procesos
5. Patrones multiproceso básicos
6. Depuración y documentación de aplicaciones multiproceso

1. Introducción a los procesos

Programación Concurrente

¿Qué es la programación concurrente?

La ***programación concurrente*** es una técnica que permite que múltiples tareas (procesos o hilos) se ejecuten de forma intercalada en un mismo sistema. Aunque no necesariamente se ejecutan al mismo tiempo, el sistema las gestiona de tal manera que parece que lo hacen simultáneamente.

Ejemplo clásico: Imagina que tienes una aplicación de chat. Mientras el usuario escribe, el sistema puede recibir mensajes, guardar el historial y actualizar la interfaz. Todo esto ocurre de forma concurrente, aunque en realidad el procesador va alternando entre tareas muy rápidamente.



1. Introducción a los procesos

Programación Concurrente

- ✓ La **programación paralela** sí implica que las tareas se ejecutan **al mismo tiempo**, pero en **distintos núcleos o procesadores**. Es decir, hay una ejecución **realmente simultánea**.
- ✓ La **programación distribuida** lleva el concepto más allá: las tareas se ejecutan en **múltiples computadoras conectadas en red**, cada una con su propio procesador y memoria.

Tipo de programación	Ejecución simultánea	Ubicación de tareas	Ejemplo práctico
Concurrente	No (intercalada)	Un solo sistema	App de chat con múltiples funciones
Paralela	Sí	Varios núcleos/procesadores	Procesamiento de imágenes
Distribuida	Sí	Múltiples computadoras	Sistema de reservas online

1. Introducción a los procesos

Concepto de procesos e hilos

- **Proceso**: Unidad de ejecución autónoma gestionada por el sistema operativo. Tiene recursos propios (memoria, espacio de direcciones, archivos abiertos, etc.) claramente aislados de otros procesos.
- **Hilo**: Unidad básica de ejecución dentro de un proceso. Varios hilos comparten recursos de un mismo proceso (espacio de memoria, archivos, etc.).

Característica	Proceso	Hilo
Memoria	Aislada	Compartida entre hilos
Recursos	Independientes	Compartidos
Comunicación	Más costosa (pipes, IPC)	Más sencilla (variables compartidas)
Gestión de errores	Más robusta	Errores pueden afectar a todos los hilos

- Usa procesos cuando necesites aislamiento y estabilidad.
- Usa hilos cuando necesites comunicación más rápida y eficiente dentro de la misma aplicación.

1. Introducción a los procesos

Ciclo de vida de un proceso en SO

Cada proceso sigue un ciclo estándar gestionado por el SO:

1. **Creación**: Se genera mediante una llamada al sistema operativo.
2. **Ejecución**: Ocupa tiempo de CPU realizando sus tareas.
3. **Espera** (Waiting): Puede estar esperando recursos (entrada/salida, datos, evento externo).
4. **Finalización** : Cuando el proceso finaliza su tarea o es terminado explícitamente.



En Java:

- **Creación:** ProcessBuilder.start()
- **Ejecución/Espera:** Process.waitFor()
- **Finalización:** Salida del método waitFor() con código de retorno.

1. Introducción a los procesos

Ciclo de vida de un proceso en SO

Este código ejecuta un proceso externo (en este caso, el comando ping) desde Java, captura su salida, espera su finalización y muestra el estado en cada etapa.

Refleja tres estados del ciclo de vida de un proceso:

- **Creación**: El proceso es lanzado.
- **Ejecución**: El proceso corre y produce salida.
- **Finalización**: El proceso termina y se obtiene su estado.

CicloProcesoSO.java

```java
package HilosVSProcesos;
import java.io.BufferedReader;

public class CicloProcesoSO {
    public static void main(String[] args) {
        try {
            System.out.println("○ Estado: Creación del proceso...");
            // Lanzamos un proceso externo (por ejemplo, ping)
            Process proceso = Runtime.getRuntime().exec("ping -n 3 www.google.com");

            System.out.println("◉ Estado: Ejecución...");
            // Capturamos la salida del proceso
            BufferedReader salida = new BufferedReader(
                new InputStreamReader(proceso.getInputStream())
            );

            String linea;
            while ((linea = salida.readLine()) != null) {
                System.out.println("📤 Salida: " + linea);
            }

            // Esperamos a que el proceso termine
            int estadoFinal = proceso.waitFor();
            System.out.println("◉ Estado: Finalización con código " + estadoFinal);

        } catch (IOException e) {
            System.out.println("✖ Error al crear o ejecutar el proceso: " + e.getMessage());
        } catch (InterruptedException e) {
            System.out.println("⏸ El proceso fue interrumpido.");
        }
    }
}
```

1. Introducción a los procesos

Diferencias entre programación paralela y distribuida:

- **Programación paralela**: La programación paralela consiste en dividir una tarea en subtareas que se ejecutan simultáneamente en múltiples núcleos o procesadores dentro de una misma máquina. Todos los hilos o procesos comparten la misma memoria física, lo que permite una comunicación rápida, pero requiere mecanismos de sincronización para evitar conflictos.

  Ventajas

  - Alta velocidad de ejecución
  - Ideal para tareas intensivas en cálculo
  - Menor latencia de comunicación

```
ExecutorService pool = Executors.newFixedThreadPool(4);
//Cada hilo procesa una parte de la imagen
```

- **Ejemplo:** Imagina que tienes que procesar una imagen de alta resolución para aplicar un filtro. Puedes dividir la imagen en 4 partes y asignar cada parte a un núcleo diferente del procesador. Todos trabajan al mismo tiempo, y al final se unen los resultados.

1. Introducción a los procesos

Diferencias entre programación paralela y distribuida:

- **Programación distribuida:** La programación distribuida implica dividir una tarea en partes que se ejecutan en diferentes máquinas conectadas por una red. Cada nodo tiene su propia memoria y CPU, y se comunican mediante mensajes. Es útil para sistemas escalables y tolerantes a fallos.

  Ventajas

  - Escalabilidad horizontal (más máquinas = más potencia)
  - Ideal para sistemas distribuidos como microservicios o big data
  - Resiliencia ante fallos de hardware

- **Ejemplo:** Supón que tienes una aplicación de análisis de datos que procesa millones de registros. Puedes distribuir esos datos entre varios servidores (nodos) y cada uno procesa su parte. Luego, los resultados se combinan

1. Introducción a los procesos

Diferencias entre programación paralela y distribuida:

¿Dónde surgen las dudas?

- ¿Qué pasa si usamos múltiples máquinas, pero con memoria compartida? ¿Es paralela o distribuida?
- ¿Y si usamos múltiples núcleos en distintas máquinas conectadas por red?
- ¿Puede una aplicación ser paralela y distribuida al mismo tiempo?

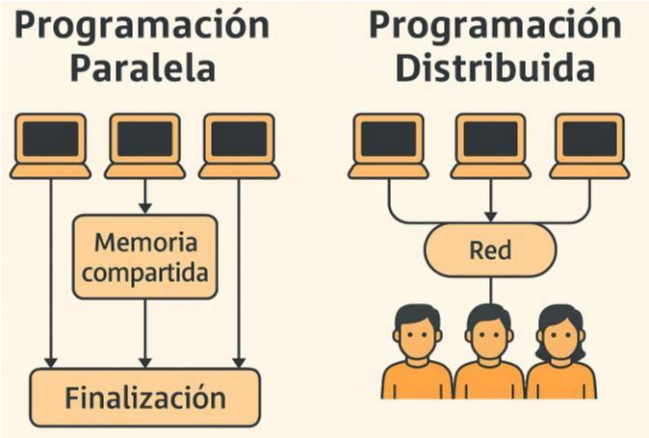


*La frontera entre ambos paradigmas se difumina en arquitecturas modernas como los **clusters**, donde se combinan técnicas de paralelismo dentro de cada nodo y distribución entre nodos.*

1. Introducción a los procesos

Diferencias entre programación paralela y distribuida:

Aspecto	Programación Paralela	Programación Distribuida
Ubicación procesos	Una máquina física	Varias máquinas en red
Comunicación	Memoria compartida o IPC	Redes (sockets, protocolos)
Objetivo principal	Rendimiento	Escalabilidad y tolerancia a fallos



2. Creación y gestión de procesos con Java

Ejecución de comandos externos (terminal) desde Java:

Java permite ejecutar comandos de la terminal directamente desde un programa, a través de las clases **Runtime** y **ProcessBuilder**. Esto facilita la integración con herramientas del sistema operativo, scripts o programas externos, desde una aplicación.

- **Runtime.getRuntime().exec()** → API clásica, más simple pero menos flexible.

- **ProcessBuilder** → Recomendado: más control sobre el entorno, directorio de trabajo, redirecciones de E/S y código de salida.

```java
package ProcesosEjemplos;
public class DemoRuntimeExec {
    public static void main(String[] args) {
        try {
            // En Windows: "notepad"
            // En Linux/Mac: "gedit" o "ls"
            Process p = Runtime.getRuntime().exec("notepad");

            // Espera a que el proceso termine
            int exitCode = p.waitFor();
            System.out.println("Proceso terminó con código: " + exitCode);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Código

2. Creación y gestión de procesos con Java

Uso básico del ProcessBuilder:

La clase **ProcessBuilder** permite ejecutar comandos externos y gestionar sus procesos desde Java. Con ella se pueden lanzar programas del sistema operativo, redirigir su salida y entrada, configurar variables de entorno y controlar su finalización. Es útil para integrar aplicaciones externas en proyectos Java o automatizar tareas desde un programa.

Ejemplo: Listar el contenido de un directorio ->

Código

```java
package ProcessBuilderEjemplo;
import java.io.BufferedReader;

public class DemoProcessBuilder {
    public static void main(String[] args) {
        try {
            // Comando a ejecutar (ejemplo: listar directorio)
            // En Windows -> "cmd", "/c", "dir"
            // En Linux/Mac -> "bash", "-c", "ls -la"
            String[] comando = {"cmd", "/c", "dir"};

            // Crear el ProcessBuilder con el comando
            ProcessBuilder pb = new ProcessBuilder(comando);

            // Iniciar el proceso
            Process proceso = pb.start();

            // Leer salida del proceso
            BufferedReader reader = new BufferedReader(
                    new InputStreamReader(proceso.getInputStream()));
            String linea;
            while ((linea = reader.readLine()) != null) {
                System.out.println(linea);
            }

            // Esperar a que termine y mostrar código de salida
            int exitCode = proceso.waitFor();
            System.out.println("Proceso finalizado con código: " + exitCode);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

2. Creación y gestión de procesos con Java

Configuración del entorno del proceso

Cuando lanzamos un proceso externo desde Java con ProcessBuilder, podemos configurar su entorno de ejecución:

- **Variables de entorno** (ej.: PATH, JAVA_HOME, personalizadas).
- **Directorio de trabajo** desde el cual se ejecutará el comando.
- **Redirección de entrada/salida** hacia consola, ficheros o streams.

Métodos útiles de la clase ProcessBuilder:



- **pb.environment()** devuelve un mapa que se puede modificar.
- **pb.directory(File)** cambia el working directory del proceso.
- **redirectOutput() y redirectError()** permiten guardar logs.
- **inheritIO()** copia directamente la salida del proceso a la consola de Eclipse.

2. Creación y gestión de procesos con Java

Ejemplos prácticos iniciales con Java y Eclipse IDE:

Ejemplo 1: Ejecutar un comando y capturar la salida

- *Ejecutar con Run (Ctrl + F11)*

Código

Notas:

- ✓ *Siempre capturar la salida (stdout/stderr) para evitar bloqueos por buffers llenos.*
- ✓ ***exitCode == 0** indica ejecución correcta; cualquier otro valor señala error.*

```java
package ProcesosEjemplos;
import java.io.BufferedReader;

public class DemoProcessBuilder1 {
    public static void main(String[] args) {
        // Crear el ProcessBuilder con el comando
        ProcessBuilder pb = new ProcessBuilder("cmd.exe", "/c", "ipconfig");

        // Combina salida estándar y de error en un solo stream
        pb.redirectErrorStream(true);

        try {
            // Inicia el proceso
            Process p = pb.start();

            // Captura la salida del proceso
            BufferedReader br = new BufferedReader(
                    new InputStreamReader(p.getInputStream()));

            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }

            // Espera a que termine el proceso
            int exitCode = p.waitFor();
            System.out.println("Proceso finalizado con código: " + exitCode);

        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

2. Creación y gestión de procesos con Java

Ejemplos prácticos iniciales con Java y Eclipse IDE:

Java permite ejecutar varios procesos externos al mismo tiempo. Cada proceso se gestiona con un objeto **Process**, y pueden correr en paralelo mientras el programa Java los controla (esperar, leer salida, terminar).

Ejemplo 2: Lanzar múltiples procesos simultáneos ->

Código

Notas:

- *Cada proceso tiene su propia salida, gestionada en un hilo lector independiente.*
- *Evita bloquear el programa principal gestionando la salida en hilos.*

```java
package ProcesosEjemplos;
import java.io.BufferedReader;

public class DemoMultiProcess {
    public static void main(String[] args) {
        // Lista de comandos a ejecutar (Windows)
        List<String[]> comandos = Arrays.asList(
            new String[]{"cmd.exe", "/c", "echo Proceso 1 && ping -n 2 127.0.0.1"},
            new String[]{"cmd.exe", "/c", "echo Proceso 2 && ping -n 3 127.0.0.1"},
            new String[]{"cmd.exe", "/c", "echo Proceso 3 && ping -n 4 127.0.0.1"}
        );

        try {
            for (String[] comando : comandos) {
                ProcessBuilder pb = new ProcessBuilder(comando);
                pb.redirectErrorStream(true);
                Process p = pb.start();

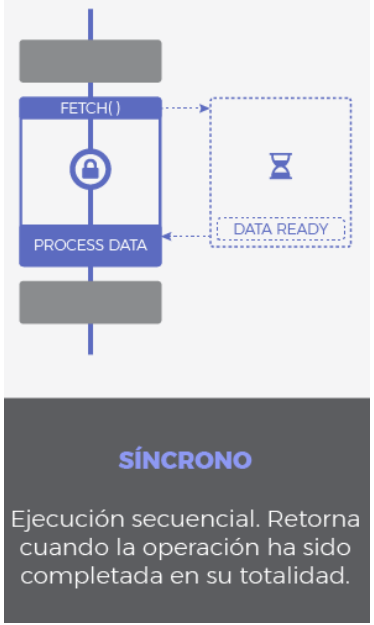
                // Cada proceso se maneja en un hilo independiente
                new Thread(() -> {
                    try (BufferedReader br = new BufferedReader(
                            new InputStreamReader(p.getInputStream()))) {
                        String linea;
                        while ((linea = br.readLine()) != null) {
                            System.out.println("[" + comando[2] + "] " + linea);
                        }
                        int exit = p.waitFor();
                        System.out.println("[" + comando[2] + "] terminado con código " + exit);
                    } catch (Exception e) {
                        e.printStackTrace();
                    }
                }).start();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

3. Comunicación entre procesos

Tipos de comunicación Síncrono vs Asíncrono

En el desarrollo de aplicaciones concurrentes, la comunicación entre procesos **(IPC)** es clave para coordinar tareas, compartir datos y garantizar la sincronización adecuada. Esta comunicación puede establecerse de forma síncrona o asincrónica, dependiendo de las necesidades del sistema:

- **Síncrona:** El proceso emisor espera una respuesta inmediata del receptor antes de continuar. Esto garantiza una secuencia controlada de ejecución, pero puede generar bloqueos si el receptor tarda en responder.
- **Asincrónica:** El emisor envía el mensaje y continúa su ejecución sin esperar respuesta. Esto mejora la eficiencia y la escalabilidad, aunque requiere mecanismos adicionales para gestionar la recepción y procesamiento de mensajes.



3. Comunicación entre procesos

Tipos de comunicación Síncrono vs Asíncrono: Ejemplo

- **Síncrono:** Imagina que tienes un programa que necesita enviar un mensaje a otro programa y esperar una respuesta antes de continuar. En este caso, el programa se detiene y espera la respuesta, como una llamada telefónica.
  - Esto puede hacer que el programa se quede "bloqueado" esperando la respuesta.

- **Asíncrono:** El programa envía el mensaje y sigue haciendo otras cosas mientras espera la respuesta. Cuando llega la respuesta, el programa la maneja sin haber estado "bloqueado".
  - Es como enviar un mensaje de texto y seguir con tu día hasta que recibes una respuesta.
  - Esto puede hacer que el programa sea más eficiente y rápido



3. Comunicación entre procesos con Java

Comunicación mediante entrada/salida estándar (stdin, stdout, stderr):

Cuando lanzamos procesos externos desde Java con **ProcessBuilder**, podemos comunicarnos con ellos a través de sus flujos de **entrada** y **salida** estándar.

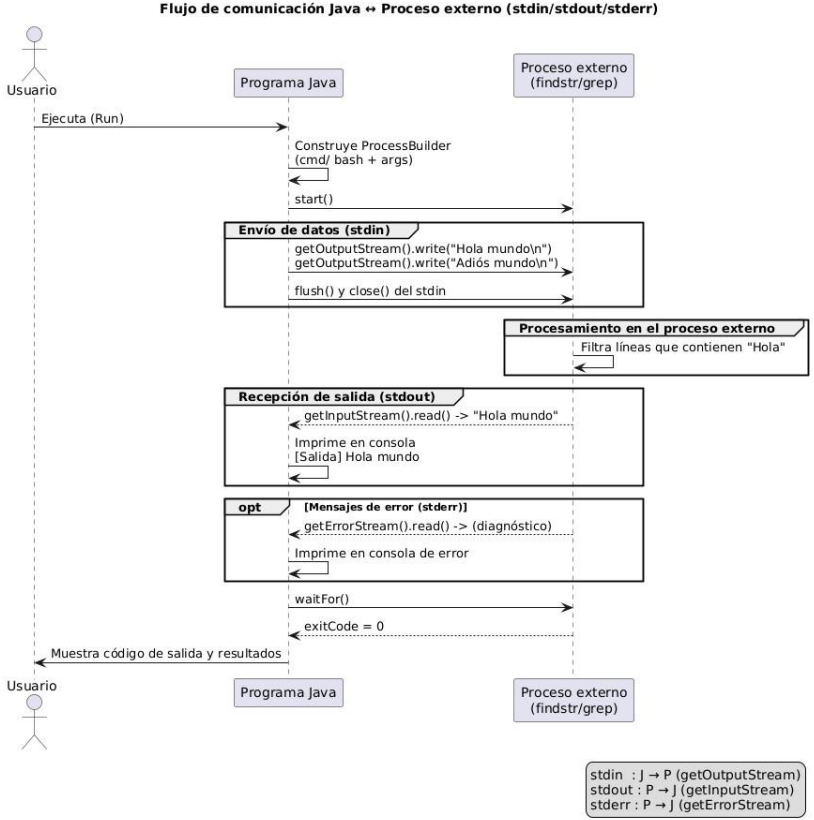
Código

Flujos de comunicación disponibles:

- **stdin (Standard Input)→ p.getOutputStream().** Permite enviar datos desde Java al proceso externo (como si el usuario escribiera en la consola).
- **stdout (Standard Output)→ p.getInputStream().** Permite recibir la salida normal generada por el proceso (mensajes, resultados).
- **stderr (Standard Error)→ p.getErrorStream().** Permite capturar errores y diagnósticos emitidos por el proceso.

3. Comunicación entre procesos con Java

Comunicación mediante entrada/salida estándar (stdin, stdout, stderr):



DemoComunicacion.java

```java
package EntradaSalida;
import java.io.*;

public class DemoComunicacion {
    public static void main(String[] args) {
        try {
            // Proceso que espera entrada: en Windows usa "findstr", en Linux/Mac usa "grep"
            String[] comando = {"cmd.exe", "/c", "findstr Hola"};
            ProcessBuilder pb = new ProcessBuilder(comando);
            Process p = pb.start();

            // ---- ENVIAR datos al stdin del proceso ----
            try (BufferedWriter writer = new BufferedWriter(
                    new OutputStreamWriter(p.getOutputStream()))) {
                writer.write("Hola mundo\n");
                writer.write("Adiós mundo\n");
                writer.flush();
            }

            // ---- LEER stdout del proceso ----
            try (BufferedReader reader = new BufferedReader(
                    new InputStreamReader(p.getInputStream()))) {
                String linea;
                while ((linea = reader.readLine()) != null) {
                    System.out.println("[Salida] " + linea);
                }
            }

            // ---- LEER stderr (si hay) ----
            try (BufferedReader err = new BufferedReader(
                    new InputStreamReader(p.getErrorStream()))) {
                String linea;
                while ((linea = err.readLine()) != null) {
                    System.err.println("[Error] " + linea);
                }
            }

            int exitCode = p.waitFor();
```

3. Comunicación entre procesos con Java

Uso práctico de pipes (tuberías) en Java:

Las **pipes (tuberías)** permiten conectar la salida de un proceso con la entrada de otro, de manera que la información fluye en cadena, igual que en la terminal.

En Java se pueden implementar de dos formas:

1. **Entre procesos externos**: redirigiendo streams con ProcessBuilder.
2. **Dentro de Java (pipes internos):** usando PipedInputStream y PipedOutputStream para comunicación entre hilos.

Nota: *Evita olvidarte de* ***cerrar streams****, o los procesos/hilos pueden quedar bloqueados esperando más datos.*



3. Comunicación entre procesos con Java

Uso práctico de pipes Internas.

Estas se usan para comunicación entre hilos dentro de la misma JVM. Son útiles cuando quieres que dos hilos intercambien datos sin usar archivos o sockets.

- **PipedInputStream y PipedOutputStream** (para datos binarios)
- **PipedReader y PipedWriter** (para datos de texto)

Cómo funcionan:

Un hilo escribe en el **PipedOutputStream**, y otro hilo lee desde el **PipedInputStream**. Ambos deben estar conectados.

```java
package EntradaSalida;
import java.io.*;

public class DemoPipesInternos {
    public static void main(String[] args) throws Exception {
        PipedOutputStream pos = new PipedOutputStream();
        PipedInputStream pis = new PipedInputStream(pos);

        // Hilo productor
        new Thread(() -> {
            try (OutputStreamWriter writer = new OutputStreamWriter(pos)) {
                writer.write("Mensaje enviado por pipe\n");
            } catch (IOException e) { e.printStackTrace(); }
        }).start();

        // Hilo consumidor
        new Thread(() -> {
            try (BufferedReader reader = new BufferedReader(new InputStreamReader(pis))) {
                String linea;
                while ((linea = reader.readLine()) != null) {
                    System.out.println("Recibido: " + linea);
                }
            } catch (IOException e) { e.printStackTrace(); }
        }).start();
    }
}
```

3. Comunicación entre procesos con Java

Uso práctico de pipes Externas.

Java no tiene pipes externas como en Unix (|) de forma nativa, pero puedes simularlas usando: **ProcessBuilder.**

Ejemplo -> ***Codigos***

```java
package EntradaSalida;
import java.io.*;

public class DemoPipesExternos {
    public static void main(String[] args) throws Exception {
        // Proceso 1: dir
        ProcessBuilder pb1 = new ProcessBuilder("cmd.exe", "/c", "dir");
        Process p1 = pb1.start();

        // Proceso 2: findstr
        ProcessBuilder pb2 = new ProcessBuilder("cmd.exe", "/c", "findstr txt");
        Process p2 = pb2.start();

        // Conectar salida de p1 → entrada de p2
        try (InputStream is = p1.getInputStream();
             OutputStream os = p2.getOutputStream()) {
            is.transferTo(os);
        }

        // Leer salida final del pipe (p2)
        try (BufferedReader br = new BufferedReader(
                new InputStreamReader(p2.getInputStream()))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        }

        p1.waitFor();
        p2.waitFor();
    }
}
```

3. Comunicación entre procesos con Java

Transmisión de datos entre procesos Java y externos (bidireccional):

Puedes interactuar de manera más compleja con procesos externos enviando y recibiendo continuamente datos.

Con **ProcessBuilder** puedes comunicarte en ambos sentidos con un proceso externo:

- **Java → proceso** por stdin (envías comandos/datos).
- **proceso → Java** por stdout (resultados) y stderr (diagnóstico).

Codigos

Nota: *Para que sea realmente full-duplex, debes leer y escribir en paralelo (hilos separados), y cerrar stdin cuando ya no vayas a enviar más datos.*

4. Sincronización y obtención de resultados de procesos

Esperar la finalización de procesos (waitFor()):

Cuando se lanza un proceso con **ProcessBuilder.start(),** Java obtiene un objeto **Process**.

Ese proceso se ejecuta de forma **asíncrona**, pero a menudo necesitamos esperar a que termine para saber si:

- Finalizó correctamente.
- Devolvió un código de salida (exit code).Debemos continuar la lógica en Java solo después de su cierre.
- El método waitFor() bloquea el hilo actual hasta que el proceso finaliza.

4. Sincronización y obtención de resultados de procesos

Esperar la finalización de procesos (waitFor()):

Ejemplo:

- **waitFor()** → bloquea indefinidamente hasta que el proceso termina.
- **waitFor(long, TimeUnit)** → espera hasta un máximo de tiempo; devuelve false si no ha terminado.
- Siempre comprobar el **exit code** (0 = correcto, otros valores indican error).
- Puede combinarse con lectura de **stdout/stderr** en paralelo mientras se espera.

DemoWaitFor.java

```java
package waitfor;
import java.io.IOException;

public class DemoWaitFor {
    public static void main(String[] args) {
        try {
            // En Windows: "ping -n 3 127.0.0.1"
            // En Linux/Mac: "ping -c 3 127.0.0.1"
            String[] comando = {"cmd.exe", "/c", "ping -n 3 127.0.0.1"};
            ProcessBuilder pb = new ProcessBuilder(comando);

            Process p = pb.start();
            System.out.println("Proceso lanzado, esperando que termine...");

            // Bloquea hasta que el proceso finalice
            int exitCode = p.waitFor();

            System.out.println("Proceso terminado con código: " + exitCode);
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Código

4. Sincronización y obtención de resultados de procesos

Gestión de resultados (salida del proceso):

Cuando un proceso externo termina, produce resultados en tres canales principales:

- **stdout** (salida estándar): datos o mensajes normales.
- **stderr** (salida de error): errores o diagnósticos.
- **exit code** (código de salida): número entero que indica éxito (0) o fallo (≠0).

Códigos

Nota: *En Java, podemos capturar estos resultados con las clases de entrada/salida (InputStream, BufferedReader) y decidir cómo tratarlos (mostrar en consola, guardar en un archivo, analizar su contenido).*

```java
package EntradaSalida;
import java.io.*;

public class DemoGestionResultados {
    public static void main(String[] args) {
        try {
            // En Windows: "ipconfig /all"
            // En Linux/Mac: "ifconfig"
            String[] comando = {"cmd.exe", "/c", "ipconfig /all"};
            ProcessBuilder pb = new ProcessBuilder(comando);

            // Combina stdout y stderr en un único stream
            pb.redirectErrorStream(true);

            Process p = pb.start();

            // Leer la salida (stdout + stderr)
            try (BufferedReader reader = new BufferedReader(
                    new InputStreamReader(p.getInputStream()))) {
                String linea;
                while ((linea = reader.readLine()) != null) {
                    System.out.println(linea);
                }
            }

            // Esperar finalización y obtener código de salida
            int exitCode = p.waitFor();
            System.out.println("Proceso finalizado con código: " + exitCode);

        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

4. Sincronización y obtención de resultados de procesos

Manejo avanzado: Procesos con tiempo de ejecución limitado:

A veces un proceso externo puede tardar demasiado o incluso quedarse bloqueado.

Para evitar que nuestro programa Java se cuelgue, podemos limitar su tiempo máximo de ejecución y, si se excede, finalizar el proceso de forma controlada.

En Java, esto se hace con:

- **waitFor(long timeout, TimeUnit unit)** → espera un tiempo máximo.
- **destroy()** → intenta terminar el proceso suavemente.
- **destroyForcibly()** → lo finaliza inmediatamente.

```java
package EntradaSalida;
import java.io.IOException;

public class DemoTimeout {
    public static void main(String[] args) {
        try {
            // Ejemplo: ping de 10 intentos (tardará varios segundos)
            String[] comando = {"cmd.exe", "/c", "ping -n 10 127.0.0.1"};
            ProcessBuilder pb = new ProcessBuilder(comando);

            Process p = pb.start();
            System.out.println("Proceso iniciado, esperando máximo 3 segundos...");

            // Espera con timeout
            if (p.waitFor(3, TimeUnit.SECONDS)) {
                int exitCode = p.exitValue();
                System.out.println("Proceso finalizó con código: " + exitCode);
            } else {
                System.out.println("Tiempo excedido, terminando proceso...");
                p.destroy(); // intento suave
                if (p.isAlive()) {
                    p.destroyForcibly(); // si no muere, forzar
                }
            }

        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

5. Patrones multiproceso básicos

¿Por qué usar patrones en los procesos?

Los patrones de diseño en concurrencia y gestión de procesos proporcionan soluciones probadas a problemas comunes. En lugar de reinventar la rueda, seguimos estructuras ya documentadas que mejoran la calidad y mantenibilidad del software.



Beneficios principales

- **Claridad y organización:** Estructuran el código, haciéndolo más fácil de entender y mantener.
- **Escalabilidad:** Permiten manejar más hilos o procesos sin reescribir toda la lógica.
- **Reutilización:** Resuelven problemas típicos (productor–consumidor, pool de hilos, pipes, etc.) de forma estándar.
- **Mantenibilidad:** Facilitan la depuración, la incorporación de nuevas funcionalidades y la colaboración en equipo.
- **Seguridad y robustez:** Reducen errores comunes en concurrencia como deadlocks, condiciones de carrera o bloqueos inesperados.
- **Portabilidad:** Aplican ideas generales (master–worker, reactor, supervisor, etc.) que sirven tanto en multihilo como en multiproceso y en distintos lenguajes.

5. Patrones multiproceso básicos

Tipos de patrones multiproceos

Patrón	Descripción	Ejemplo práctico
Process Pool / Prefork	Mantener un grupo fijo de procesos listos para atender peticiones. Evita coste de crear un proceso por cada solicitud.	Servidores web (Apache prefork, Nginx workers).
Supervisor – Worker	Un proceso maestro monitoriza y reinicia procesos “worker” que fallen. Proporciona tolerancia a fallos.	Sistemas de colas (RabbitMQ, Celery con supervisores).
Pipeline	Encadenar procesos de manera que la salida de uno sea la entrada del siguiente.	`ls
Map–Reduce / Batch	Dividir trabajo en procesos independientes (map), luego reunir resultados (reduce).	Hadoop, Spark en modo multiproceso.
Actor / Microservicio	Cada proceso actúa como “actor” aislado que se comunica por mensajes.	Akka (modo multiproceso), microservicios con colas.
Trabajadores en cola (Work Queue)	Varios procesos consumidores extraen tareas de una cola compartida.	RabbitMQ con múltiples consumidores, Celery workers.
Master – Worker distribuido	Un proceso maestro reparte trabajos a múltiples procesos en diferentes nodos.	MPI (Message Passing Interface), clusters de renderizado.

5. Patrones multiproceso básicos

Patrón Master-Worker:

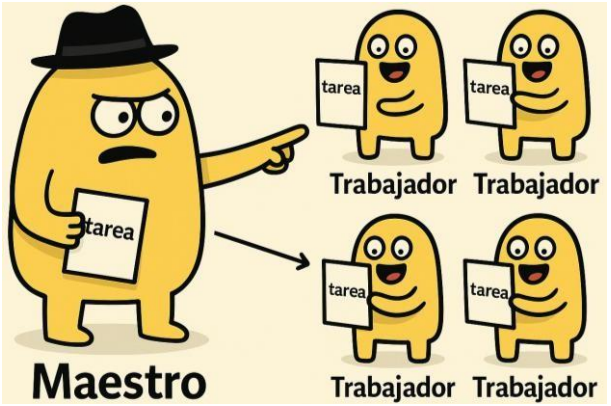
Es un patrón habitual en programación paralela, donde hay un proceso principal (Master) que distribuye tareas a varios procesos secundarios (Workers).

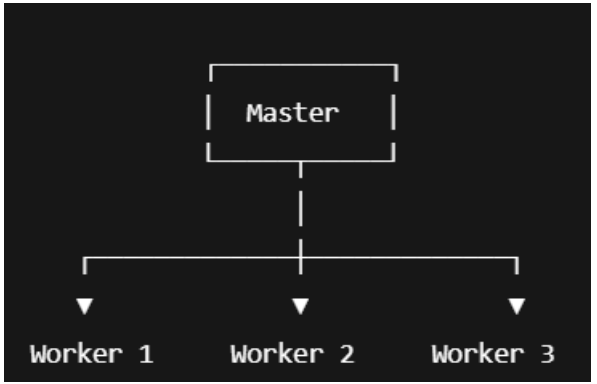
Características:



- **Master:** Asigna tareas y recoge resultados.
- **Workers:** Procesan tareas asignadas y retornan resultados al Master.



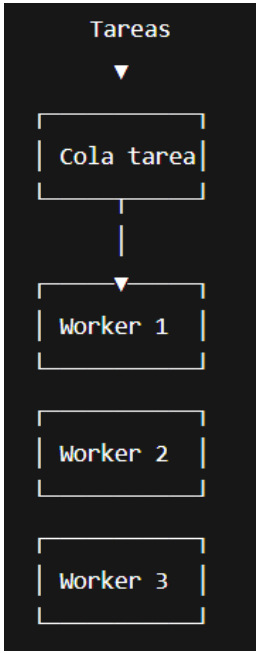




5. Patrones multiproceso básicos

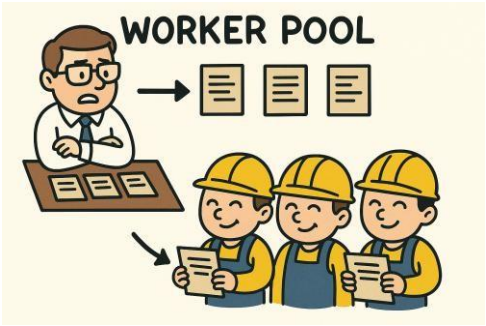
Patrón Worker Pool

Es una variante del **master-worker** en la que disponemos de un grupo (pool) de workers preinicializados, que procesan tareas a medida que llegan. Ideal para gestionar múltiples tareas usando un número limitado y controlado de procesos.



Características:

- Workers pre-creados y reutilizados (mejora rendimiento).
- Cola de tareas que los workers consumen.



6. Depuración y documentación de aplicaciones multiproceso

Uso del depurador integrado en Eclipse IDE para procesos externos

Eclipse permite depurar fácilmente procesos Java externos:

1. Define puntos de ruptura (breakpoints) en Eclipse.
2. Ejecuta tu aplicación en modo depuración (Debug as → Java Application).
3. Controla ejecución usando comandos:
    - **F5 (Step Into):** entrar en métodos.
    - **F6 (Step Over):** avanzar línea a línea sin entrar.
    - **F7 (Step Return):** salir del método actual.
    - **F8 (Resume):** continuar ejecución hasta el siguiente breakpoint.

***Nota**: La depuración de procesos externos (no Java) es limitada, por lo que es esencial capturar y mostrar claramente stdout y stderr para depuración.*

6. Depuración y documentación de aplicaciones multiproceso

Gestión de errores y excepciones en ejecución de procesos:

Cuando trabajamos con procesos externos en Java (por ejemplo, ejecutar comandos del sistema operativo), pueden surgir errores como rutas incorrectas, permisos insuficientes o comandos inexistentes. Para evitar que el programa se detenga abruptamente, usamos bloques try-catch que permiten capturar excepciones y mostrar mensajes útiles al usuario.

Ejemplo:

LanzadorDeProcesos.java

```java
package ejemplos;
import java.io.IOException;

public class LanzadorDeProcesos {
    public static void main(String[] args) {
        try {
            // Intentamos ejecutar un comando del sistema
            Process proceso = Runtime.getRuntime().exec("notepad.exe");
            System.out.println("☑ Proceso iniciado correctamente.");
        } catch (IOException e) {
            System.out.println("✕ Error al intentar ejecutar el proceso.");
            System.out.println("Detalles técnicos: " + e.getMessage());
        } catch (SecurityException se) {
            System.out.println("⦸ Permiso denegado para ejecutar el proceso.");
        } catch (Exception ex) {
            System.out.println("⚠ Se produjo un error inesperado.");
            ex.printStackTrace();
        }
    }
}
```

6. Depuración y documentación de aplicaciones multiproceso

Gestión de errores y excepciones en ejecución de procesos:

¿Qué hace el código?

- Intenta ejecutar el bloc de notas (notepad.exe) en Windows.
- Si el comando falla (por ejemplo, en otro sistema operativo), se captura la excepción IOException.
- También se contempla SecurityException por si el entorno bloquea la ejecución.
- Se imprime un mensaje claro para cada tipo de error.

Miguel Soriano

msoriano@ucam.edu

Instituto Superior de Formación Profesional
San Antonio de Murcia



© FP SAN ANTONIO

FP SAN ANTONIO
INSTITUTO SUPERIOR DE FORMACIÓN PROFESIONAL